

Roll No: 2003055

Set - Y

Lab Performance Evaluation 1 + Lab Final

Lab Task Q[1]

Question:

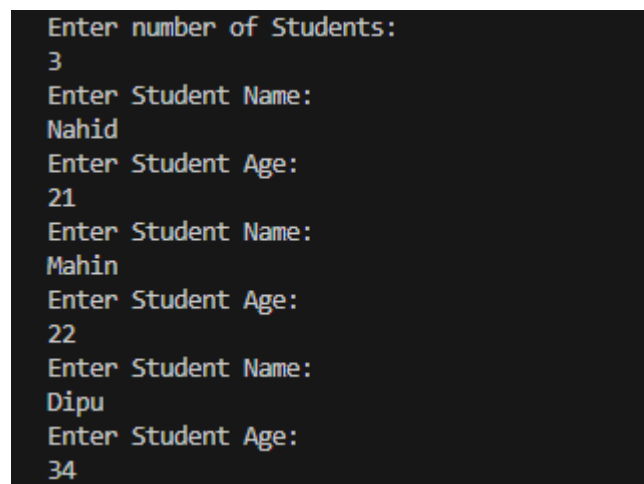
Enter Number of Students, then input each of their name and age and store them in students.txt file

Solution (Code/Add all circuits diagrams):

```
touch "students.txt"
echo "Enter number of Students: "
read n

for((i=1;i<=n;i++));do
    echo "Enter Student Name:"
    read name
    echo "Enter Student Age: "
    read age
    echo "$name $age" >> students.txt
done
```

Output/Simulation Result (Screenshot/SnapShot):



```
Enter number of Students:
3
Enter Student Name:
Nahid
Enter Student Age:
21
Enter Student Name:
Mahin
Enter Student Age:
22
Enter Student Name:
Dipu
Enter Student Age:
34
```

 students.txt

1 Nahid 21

2 Mahin 22

3 Dipu 34

4 |

Lab Task Q[2]

Question:

Create a child, then 2 more child inside that child. Replace the child with a function.

Solution (Code/Add all circuits diagrams):

Program.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2, pid3;

    // Create a new child process
    pid1 = fork();

    if (pid1 < 0) {
        // Fork failed
        printf("Error: fork failed\n");
        exit(EXIT_FAILURE);
    } else if (pid1 == 0) {
        pid2 = fork();
        if (pid2 < 0) {
            // Fork failed
            printf("Error: fork failed\n");
            exit(EXIT_FAILURE);
        } else if (pid2 == 0) {
            usleep(300 * 1000);
            printf("GrandChild1, pid= %d,
ppid=%d\n", getpid(), getppid());
            exit(EXIT_SUCCESS);
        }
        else{
            pid3 = fork();
            if (pid3 < 0) {
                // Fork failed
                printf("Error: fork failed\n");
                exit(EXIT_FAILURE);
            } else if (pid3 == 0) {
                usleep(300 * 1000);
                printf("GrandChild2, pid= %d,
ppid=%d\n", getpid(), getppid());
```

```

        exit(EXIT_SUCCESS);
    }
    else{
        usleep(200 * 1000);
        printf("Child process: Replacing program
with execv...\n");
        char *program = "./hello_world";
        char *args[] = {program, NULL};
        if (execv(program, args) == -1) {
            printf("Error: execv failed\n");
            exit(EXIT_FAILURE);
        }
    }

}

else{
    usleep(300 * 1000);
    printf("Parent, pid= %d, ppid=%d\n", getpid(),
getppid());

}

return 0;
}

```

Hello_world.c:

```
#include <stdio.h>
```

```

int main() {
    printf("Hello World\n");
    return 0;
}

```

Output/Simulation Result (Screenshot/SnapShot):

```

Child process: Replacing program with execv...
Hello World
Parent, pid= 6845, ppid=1357
GrandChild1, pid= 6847, ppid=478
GrandChild2, pid= 6848, ppid=478

```

Lab Task Q[3]

Question:

Two process P1 and P2 will be created. Two universal variable x and y = 0. P1 increase x by 2 and P2 increase x by 1 with maintaining semaphore and without semaphore P1 increase y value by 1 and P2 increase y value by 2. All process ends when x = 6.

Solution (Code/Add all circuits diagrams):

```
#include <xinu.h>

void P1(sid32 consumed, sid32 produced);
void P2(sid32 consumed, sid32 produced);

int32 x = 0;
int32 y = 0;

int xsh_sem()
{
    sid32 produced, consumed;

    consumed = semcreate(1); // P2 runs first
    produced = semcreate(0); // P1 waits first

    resume(create(P1, 1024, 20, "P1", 2, consumed, produced));
    resume(create(P2, 1024, 20, "P2", 2, consumed, produced));

    return 0;
}

void P2(sid32 consumed, sid32 produced)
{
    while (1) {
        wait(consumed);

        if (x >= 6) {
            signal(produced);
            break;
        }

        x += 1;
        printf("P2: x = %d\n", x);

        if (x >= 6) {
            signal(produced);
            break;
        }

        signal(produced);

        y += 2;
        printf("P2: y = %d\n", y);
    }
}
```

```

void P1(sid32 consumed, sid32 produced)
{
    while (1) {
        wait(produced);

        if (x >= 6) {
            signal(consumed);
            break;
        }

        x += 2;
        printf("P1: x = %d\n", x);

        if (x >= 6) {
            signal(consumed);
            break;
        }

        signal(consumed);

        y += 1;
        printf("P1: y = %d\n", y);
    }
}

```

Output/Simulation Result (Screenshot/SnapShot):

```

-----
XINU
-----

Welcome to Xinu!

xsh $ semrun
P2: x = 1
xsh $ P1: x = 3
P1: y = 1
P2: y = 3
P2: x = 4
P1: x = 6
P2: y = 5
xsh $

```